II) Creating & Writing Delta Tables

A) create & write

B) write modes, schema enforcement, envolution

C) Partitioned Delta Tables

D) Contraints in Delta

E) generated columns

F) Identity columns

G) Metadata Inspection - Describe Details (&) History

H) small file handling During writes

SECTION 2 - Creating & Writing Delta Tables

A)

```
df.write.format('delta')\
        .save('/mnt/date/sales')
```

(unmanaged)

```
spark.read.format('delta')\
        .load('/mnt/date/sales')
```

- Data stored exactly at given path.
- No live metastore registration
- spark doesn't own the location

Managed Delta Table: (metastore-Backed)

```
df.write.format('delta')
        .saveAsTable('sales_table')
```

Now spark:

- creates a table in metastore
- stores data in warehouse directory

Default location:

```
/user/hive/warehouse/sales_table/
```

---

just like { Managed Tables vs External Tables } (Nothing more than that.)

Converting parquet to Delta:

```
DeltaTable.convertToDelta(spark,
                          "parquet.'/mnt/data/sales'")
```

What happens if you write without format('delta')?

```
df.write.save("/mnt/data/sales")
```

- Default format might be parquet
- NO _delta_log

Registering Existing Delta Path as Table

```
spark.sql("""
        CREATE TABLE sales_table
        USING DELTA
        LOCATION '/mnt/data/sales'
        """)
```

B) Write modes & schema enforcement :

append
overwrite
errorIfExists
ignore

```
df.write.format('delta')\
  .mode('mode_name')\
  .save('—')
```

append	• Add new data • creates new version • old data, remains
overwrite	• mark old files as removed • Adds new files • New version created
ErrorIfExists	• If table exists write fails
ignore	• If table exists ↳ writes Ignored ↳ no error.

production Tip : overwrite partition only

↳ Instead of overwriting whole Table;
overwrite only matching partitions.

```
df.write.format('delta')\
  .mode('overwrite')\
  .option("replaceWhere",
          "date = '2024-01-01'")\
  .save("/mnt/sales")
```

Schema Enforcement :

Existing table schema:

id INT
name STRING

Now someone writes:

id STRING
name STRING
age INT

Delta:

- will compares schema
- reject writes

Delta validates incoming data schema against table schema Before commit

Schema Evolution :

(Allow table schema to change safely)

Existing table schema:

id INT
name STRING

Now someone writes:

id INT
name STRING
age INT

⇒ By default: Delta rejects this

⇒ Here we are updating the schema:

so, use: `.option('mergeSchema', 'true')`

Feature	Schema Enforcement	Schema Evolution
Purpose	Protect schema	Allow safe change
Default	Enabled	Disabled
Prevents corruption	Yes	Yes
Adds new column	NO	Yes
Changes datatype	No	Limited (mostly gives error)

Auto Merge settings

```
spark.conf.set("spark.databricks.delta.schema
                .autoMerge.enabled", "true")
```

⇒ Now (mergeSchema) automatically applied.

---

c) Partitioned Delta Tables

```
df.write.format('delta')\
  .partitionBy('date')\
  .save('/mnt/sales')
```

- creates folders
- helps with equality filters.

Z-ordering:

- reorders data within files
- helps multi-column filtering.

replacewhere + partition (Safety)

```
df.write.format('delta')\
  .mode('overwrite')\
  .option('replaceWhere',
          "date = '2024-01-01'")\
  .save('/mnt/sales')
```

D) Constraints & Data Quality Enforcement in Delta

Types of Constraints in Delta:

Delta supports:

1. NOT NULL
2. Check constraints

① Not Null Constraints:

```
create Table sales(
        id INT NOT NULL,
        amt Double,
) using Delta;
```

Now if someone tries to enter null ids gives error.

② Check constraint

```
create Table sales (
        id INT,
        amt Double,
        Constraint valid_amt Check (amt > 0)
) using Delta;
```

③ Adding constraint later:

```
ALTER Table
ADD CONSTRAINT post_amt check (amt>0)
```

Constraint definition stored in:
- delta_log JSON metadata section

5) Generated Columns :

#(production optimization Feature)

Generated Columns
=
Columns automatically computed from other columns and enforced by Delta

```
create Table sales (
    ord_id INT,
    ord_ts Timestamp,
    ord_date Date Generated Always
        As ( cast ( order_ts, AS DATE))
) using Delta,
```

Here; (ord_date is generated from ord_ts
user does not provide ord_date)

Generated Column + Partitioning :

```
create table sales (
    ord_id INT,
    ord_ts timestamp,
    ord_date Date Generated Always AS
        (cast (ord_ts AS DATE))
)
using Delta
PartitionedBy (order_date);
```

f) Identity Columns:

#(Auto Increment in Delta)

Identity Columns

= A column where Delta automatically generates sequential numbers.

↓

Like, auto-increment in SQL databases.

```
create Table customers (
    cust_id BigInt Generated Always As Identity,
    name string,
    country string
) using Delta;
```

Generated Always (vs) Generated By Default

Generated Always	Generated By Default
• User cannot provide value manually	• User can provide value manually. If not provided → Delta generates.
Insert Into cust values (100, 'A', 'xyx') → error.	

(5) Metadata Inspection
- Describe Detail & History

① Describe Detail :

SQL :
```
Describe Detail sales;
```

pyspark :
```
spark.sql("Describe Detail sales").show(truncate=fals
```

↓

(version timestamp operation ....)

what it shows?

column	meaning
format	delta
id	Table UUID
location	storage path
sizeInBytes	Total data size
numFiles	Number of data files
partitionColumns	Partition columns
CreatedAt	creation timestamp
lastModified	Last updated time
minReaderVersion	Delta protocol version
minWriterVersion	Delta protocol version

version 2
operation = write
mode = overwrite

history is stored in
↓
_delta_log/*.json

```
Describe Extended sales;
Describe Detail sales;
Describe History sales;
```
→ Find accidental Delete

Inspecting Table properties :
↓
```
Show TBLPROPERTIES sales;
```

H) small File handling During Writes:

① optimizeWrite :

```
spark.conf.set("spark.databricks.delta.optimizeWrite
.enabled", "true")
```

② autoCompact :

```
spark.conf.set("spark.databricks.delta.autoCompact
.enabled", "true")
```

③ manual Compaction:

```
optimize sales;
```